

Lab1. Introducción a R

Bioinformática. Ingeniería Biomédica

Empezando con R

Instalando R

- Primero instalamos R (<http://cran.r-project.org>)
- Una vez instalado R base podemos instalar algún paquete adicional. Para ello necesitamos disponer de conexión a internet para acceder a los repositorios. Básicamente vamos a trabajar con dos repositorios: CRAN (https://cran.r-project.org/web/packages/available_packages_by_name.html) o Bioconductor (<http://www.bioconductor.org>).

Cómo trabajar en R

- Con el editor de R. El propio programa lleva un editor incorporado.
- RStudio (<http://rstudio.org/>). Es una interfaz que incorpora el editor, la consola de R, acceso a paquetes, ventana de gráficos, historia de instrucciones, etc.

Instalando paquetes de R

Podemos instalar un paquete de R de varias formas:

- Desde la consola de R con la función `install.packages()`:

```
install.packages("FMM")
```

El comando `install.packages()` suele pedir al usuario que elija un ‘espejo’ del repositorio. También va a comprobar las dependencias del paquete a instalar, siempre que el argumento `dependencies = TRUE` (valor por defecto), y la compatibilidad del paquete con la versión de R que estemos manejando.

- Desde un directorio local en el que tenemos el código fuente del paquete en un archivo:

```
install.packages("path/to/mypackage.tar.gz", repos = NULL, type="source")
```

El fichero tendrá una extensión `.tgz/.zip`.

Para comprobar si tenemos instalado un paquete en R podemos utilizar el comando:

```
library()
```

Cargando paquetes en R

Por defecto, al iniciar una sesión de R se cargan unos paquetes básicos. Los paquetes por defecto en nuestra sesión de R son:

```
getOption("defaultPackages")
```

```
## [1] "datasets" "utils"      "grDevices" "graphics"  "stats"     "methods"
```

Para cargar un paquete específico utilizamos el comando,

```
library("FMM")
```

Al cargar un paquete se importan todas las funciones de dicha librería a nuestra sesión de R. Para consultar los paquetes cargados en nuestra sesión de R podemos utilizar,

```
print(.packages())
```

```
## [1] "stats"      "graphics"  "grDevices" "utils"      "datasets"  "methods"
## [7] "base"
```

Alternativamente podemos utilizar el comando `sessionInfo()`, que nos proporciona detalles de nuestra sesión.

Ayuda en R

Se puede obtener ayuda de R de varias formas:

```
help.start()           # enlaces a material de ayuda
help(sum)              # para acceder a los ficheros de ayuda de la función sum
?sum                  # para acceder a los ficheros de ayuda de la función sum
example(sum)           # uso de la función sum a través de un ejemplo
help.search("sum")      # búsqueda de ficheros de ayuda que contienen sum
help(package="FMM")     # ayuda del paquete FMM
```

Analizando una sola variable

El vector es la estructura de datos más sencilla en R. Un **vector** es una colección de uno o más datos del mismo tipo.

R distingue entre minusc y mayusc !!!

Construyendo y jugando con un vector

Supongamos que estamos estudiando la expresión de 10 genes en una muestra y los valores observados son,

```
x <- c(3,6,3,6,7,1,4,7,4,4)
```

Con la función `c()`, hemos creado un vector que almacenamos en un objeto llamado `x` con el operador `<-`,

```
class(x)
```

```
## [1] "numeric"
```

que es un vector formado por números.

Podemos ver su contenido completo:

```
x
```

```
## [1] 3 6 3 6 7 1 4 7 4 4
```

1. Para acceder a elementos que ocupan posiciones específicas en un vector podemos utilizar `[]`.

a) Obtén el primer elemento de este vector de datos y el que ocupa la posición 7.

Cuando ponemos dos enteros separados por `:` nos devuelve la secuencia de todos los enteros entre el primero y el segundo de 1 en 1.

```
1:3
```

```
## [1] 1 2 3
```

podríamos usar alternativamente `seq(1,3,by=1)`.

b) Obtén los elementos del vector `x` entre las posiciones 3 y 7.

c) Podemos utilizar la función `c()` para indicar entre `[]` las posiciones que queremos recuperar. Obtén los valores de `x` que ocupan las posiciones 1, 3, y de la 5 a la 8.

2. Puede que nuestro interés en ver los datos no venga dado por la posición que ocupan sino por su valor. Por ejemplo, queremos saber cuántos de los elementos del vector cumplen una condición, por ejemplo su valor es mayor o igual a 4:

```
x[x >= 4]
```

```
## [1] 6 6 7 4 7 4 4
```

Devuelve un vector de `TRUE/FALSE` con `TRUE` en las posiciones que verifican la condición.

Se pueden construir condiciones complejas utilizando los operadores `&` (and) y `|` (or).

a) Obten los valores mayores que 4 y menores o iguales que 6,

b) Ahora los valores que son menores o iguales a 4 o mayores que 6.

c) ¿Y los que son iguales a 4? Utiliza el operador `==` (igual a).

d) Podemos obtener las posiciones del vector que cumplen una condición con la función `which()`, indicando como argumento la condición a cumplir. Obtén las posiciones de los valores iguales a 4.

Ordenando un vector

Para ordenar el vector `x`,

```
sort(x)
```

```
## [1] 1 3 3 4 4 4 6 6 7 7
```

nos devuelve los valores ordenados. Con frecuencia necesitamos saber la posición que ocupaban en el vector original estos valores,

```
sort(x, index.return = TRUE)
```

```
## $x
```

```
## [1] 1 3 3 4 4 4 6 6 7 7
```

```
##
```

```
## $ix
```

```
## [1] 6 1 3 7 9 10 2 4 5 8
```

3. Utiliza la función `sort()` con el argumento `decreasing = TRUE`, ¿qué indica este argumento?

[los ordena de manera decreciente](#)

Generando observaciones de distribuciones de probabilidad

Para obtener ayuda de las distintas distribuciones de probabilidad de la que podemos generar observaciones, `?Distributions`

Para generar 100 observaciones de una distribución normal con media 1 y desviación estándar 0.1 utilizamos,

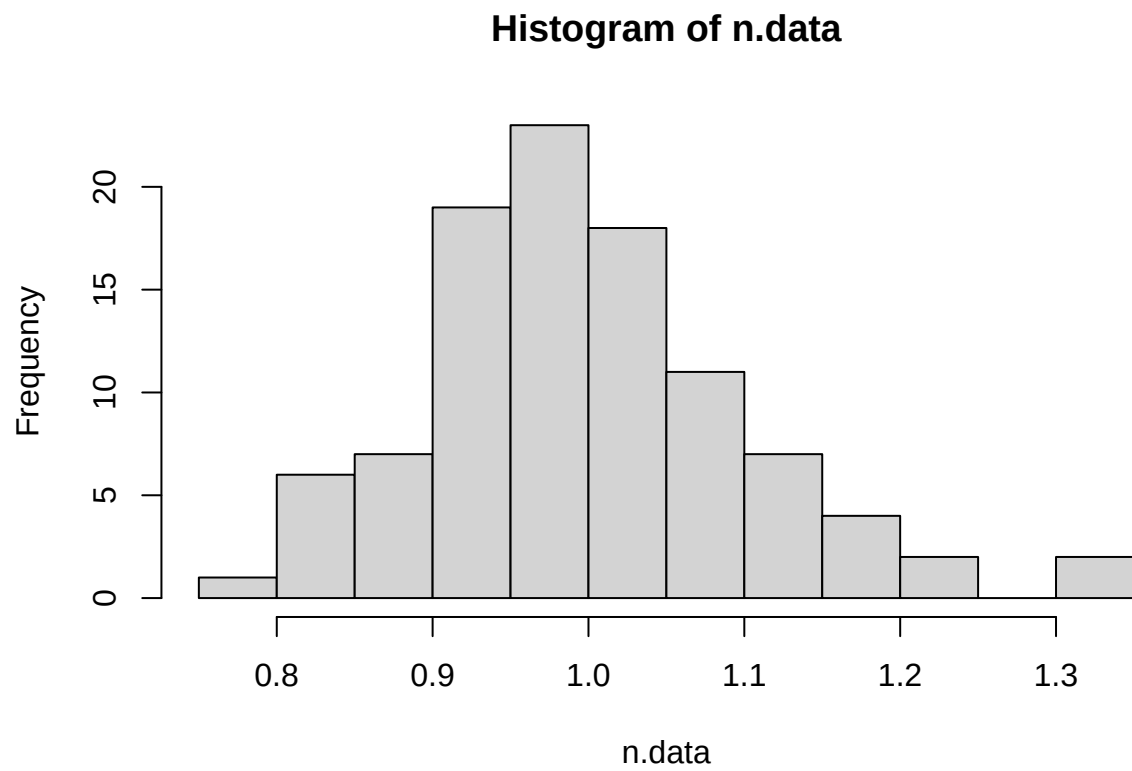
```
n.data <- rnorm(n=100, mean=1, sd=0.1)
```

```
head(n.data) # visualizamos los primeros elementos
```

```
## [1] 0.8507576 0.9850301 0.9886528 1.0524127 1.0498240 0.9085347
```

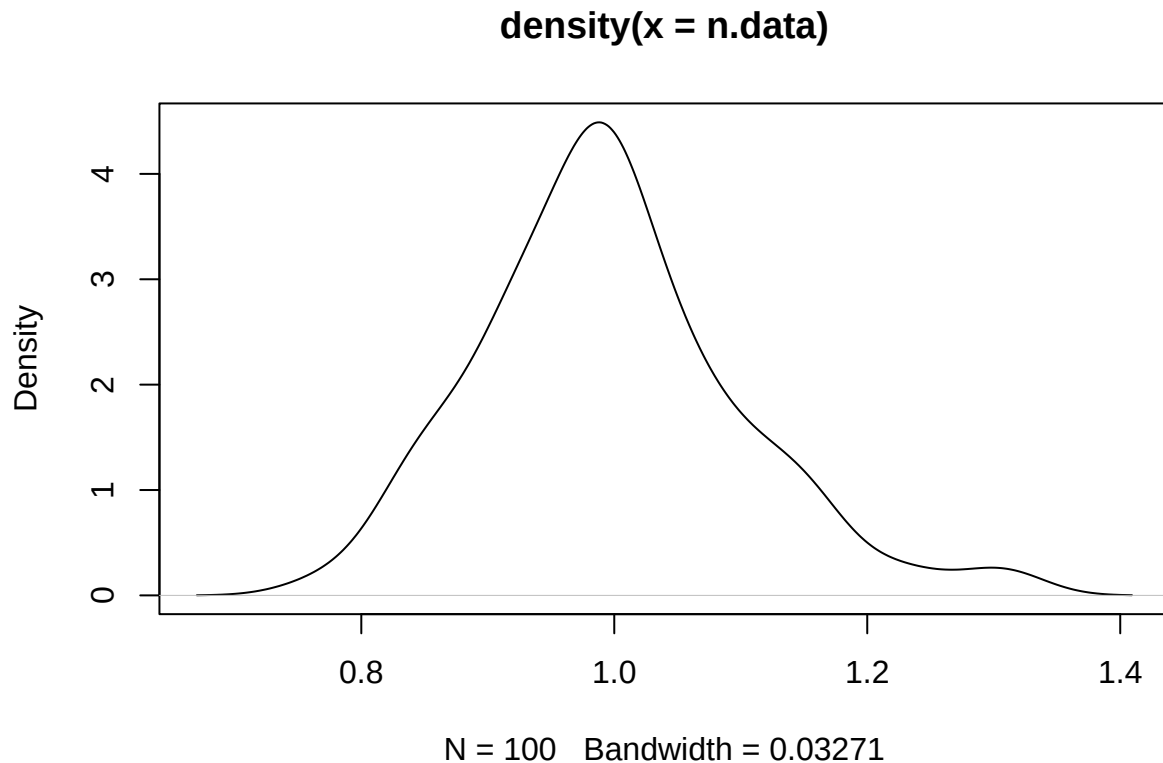
Podemos representar estos datos en un histograma,

```
hist(n.data)
```



u observar la forma de su función de densidad,

```
plot(density(n.data))
```



La función `rnorm()` tiene tres argumentos: `n` (el número de observaciones a generar), `mean` (la media de la distribución) y `sd` (su desviación típica). Para generar observaciones de otras distribuciones utilizamos el prefijo `r` añadido al nombre de la distribución. Otros prefijos, `p`, `q` y `d`, se utilizan para trabajar con probabilidad, cuantiles y densidades, respectivamente.

Matrices

Supongamos que tenemos la expresión 10 genes en 4 muestras. Simulamos estos datos de una distribución de Poisson.

```
set.seed(123)
x <- matrix(rpois(10*4,lambda=5),nrow=10)
x
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    4    9    8    9
## [2,]    7    5    6    8
## [3,]    4    6    6    6
## [4,]    8    5   11    7
## [5,]    9    2    6    1
## [6,]    2    8    6    5
## [7,]    5    3    5    6
## [8,]    8    2    5    3
## [9,]    5    4    4    4
## [10,]   5    9    3    3
```

La función `matrix()` crea la matriz con `ncol` columnas y `nrow` filas colocando los datos por columnas. Si alguno de estos dos argumentos no se especifica, adaptará las dimensiones de la matriz al vector que le

pasamos como argumento. Un argumento interesante de `matrix()` es `byrow=TRUE` que sirve para que las observaciones se coloquen por filas en lugar de por columnas,

```
matrix(1:9,nrow=3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
matrix(1:9,nrow=3,byrow=TRUE)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

4. ¿Para que sirve el comando `set.seed(123)`?

5. ¿Qué tipo de objeto es `x`?

Las funciones `colnames()` y `rownames()` sirven para dar nombres a columnas y filas respectivamente. Podríamos ejecutar,

```
colnames(x) <- c("sample1","sample2","sample3","sample4")
rownames(x) <- c("gene1","gene2","gene3","gene4","gene5","gene6","gene7","gene8","gene9","gene10")
```

6. Utiliza estas funciones junto con `paste0()` para hacer asignar nombres a las filas y columnas.

Podemos acceder al contenido de la matriz con `[,]`. Por ejemplo, la expresión del gen en la fila 6 y en la columna 2,

```
x[6,2]
```

```
## [1] 8
```

o bien utilizando los nombres con:

```
x["gene6","sample2"]
```

7. ¿Y la expresión de todos los genes en la muestra 2?

Listas

Las listas son muy importantes en R. Supongamos que estamos trabajando con un grupo de 10 genes que representamos con un número, es decir que nuestro universo de genes es el conjunto $1, \dots, 10$. Supongamos que se han identificado 3 grupos de genes (genesets),

```
geneset1 <- c(1,4)
geneset2 <- c(2,5,7,8)
geneset3 <- c(1,5,6)
```

La mejor manera de almacenar esta información es en una lista. Para crear una lista vacía,

```
(genesets <- vector("list",3))
```

```
## [[1]]
## NULL
##
## [[2]]
## NULL
```

```
##  
## [[3]]  
## NULL
```

Observa los paréntesis en la instrucción anterior!

Asignamos cada elemento de la lista a cada conjunto de genes,

```
genesets[[1]] <- geneset1  
genesets[[2]] <- geneset2  
genesets[[3]] <- geneset3
```

```
genesets  
  
## [[1]]  
## [1] 1 4  
##  
## [[2]]  
## [1] 2 5 7 8  
##  
## [[3]]  
## [1] 1 5 6
```

Para dar un nombre a cada uno de los elementos de la lista:

```
names(genesets) <- paste0("geneset",1:3)  
genesets
```

```
## $geneset1  
## [1] 1 4  
##  
## $geneset2  
## [1] 2 5 7 8  
##  
## $geneset3  
## [1] 1 5 6
```

lapply y sapply

En las listas de R cada elemento puede tener una longitud distinta. Para hacer operaciones dentro de cada elemento de una lista podemos utilizar la función `lapply()`, cuya sintaxis es,

```
lapply(x,FUN)
```

donde `x` es un lista y `FUN` la función que queremos aplicar a cada elemento de la lista.

Así, por ejemplo para averiguar la longitud que tiene cada grupo de genes podemos utilizar,

```
lapply(genesets,FUN=length)
```

```
## $geneset1  
## [1] 2  
##  
## $geneset2  
## [1] 4  
##  
## $geneset3  
## [1] 3
```

El resultado es una lista con tantos elementos como `geneset` y en cada uno de ellos se almacena la longitud del vector correspondiente.

8. ¿Qué ocurre si en lugar de utilizar `lapply()` utilizamos `sapply()`?

Leyendo y escribiendo datos

Podemos cargar datos en nuestro entorno de R directamente, cargando un objeto externo de R (extensiones `.rda` o `.RData`) o cargando un objeto interno de R, bien sea de un paquete o de un fichero `.txt`, `.csv` o excel:

Cargar un conjunto de datos interno. Vamos a utilizar el conjunto de datos `iris` disponible con los paquetes de R básicos. En este conjunto de datos se cuantifican las características morfológicas de tres especies de flores de iris.

```
data(iris)
```

Podéis consultar sus características con la instrucción `?iris`.

```
load(file="mydata.RData")
```

Cargar datos externos, Para guardar un objeto de datos:

```
save(DATA, file="myData.RData")
```

Leer datos Leer datos en forma de tabla en un fichero `.dat`,

```
mydata <- read.table("file.dat", header = TRUE, sep="\t", row.names = 1)
```

Si tenemos los datos en un fichero `.csv`,

```
mydata <- read.csv("mydata.csv")
```

Para escribir esos `data frames` o tablas en ficheros,

```
write.table(x, file = "myexcel.xls", sep = " ")
write.csv(x, col.names = NA, sep = ",")
```

También es posible leer datos de un fichero excel. Existen varios paquetes disponibles, por ejemplo `xlsx`,

```
install.packages("xlsx", dependencies=TRUE)
library(xlsx)
mydata <- read.xls("mydata.xls")
```

NOTA: Las funciones `read.csv` o `write.csv` tienen como argumento el nombre de un fichero, que, por defecto y siempre que no se le pase como nombre de fichero la ruta completa, R buscará en el directorio de trabajo actual. Para encontrar la ruta actual del directorio de trabajo,

```
getwd()
```

```
## [1] "D:/Itziar/Clases/Ingenieria Biomedica/Bioinformatica/Material/Laboratorios/Lab1 IntroR"
```

Para cambiar este directorio,

```
setwd("nueva/ruta")
```


Seleccionando datos

Los datos cargados existen en R como objetos de tipo `data.frames`, que son estructuras de tipo tabla donde las columnas pueden contener datos de distintos tipos.

9. Una vez cargados los datos ‘iris’ en la sección anterior, utiliza la función `str()` para conocer su estructura. Guardamos los datos relevantes en un nuevo objeto de tipo `data.frame`,

```
myiris <- data.frame(Sepal.Length = iris$Sepal.Length, Sepal.Width = iris$Sepal.Width, Species = iris$Species)
```

El signo `$` después del nombre del data frame y seguido de un nombre de columna devuelve los datos en esa columna.

10. Además de por el nombre, podemos acceder a las columnas del `data.frame` por la posición que ocupan utilizando la sintaxis `[,]`.

a) Obtén las variables relativas a la longitud y anchura del sépalo, junto con la especie.

b) Obtén todas las variables salvo la última.

11. Las funciones `cbind()` y `rbind()` permiten añadir columnas y filas respectivamente.

a) Añade una variable llamada `Stalk.Length` generada como 30 observaciones de una $N(0, 0.1)$, 30 de una $N(1.3, 0.1)$, 30 de una $N(1.8, 0.1)$ y otras 30 de una $N(2, 0.1)$.

b) Añade un nuevo individuo con la información: Longitud de sépalo (`Sepal.Length`) 10.1, anchura de sépalo (`Sepal.Width`) 0.5, longitud de pétalo (`Petal.Length`) 2.5, anchura de pétalo (`Petal.Width`) 0.9 y especie (`Species`) “myspecies”.

12. La sintaxis `subset(x, condicion)` permite seleccionar un subconjunto del `data.frame` `x` que cumple la `condicion`. Para definir la condición es posible utilizar cualquiera de los operadores relacionales, o evaluar más de una condición con los operados `&` (Y), `|` (O) y `!` (NO). Selecciona los individuos de `iris` cuya longitud de sépalo es mayor que 7.5.

Operaciones estadísticas básicas

R tiene implementadas numerosas funciones que permiten llevar a cabo análisis estadísticos básicos.

13. ¿Cuál es el resultado de utilizar la función `summary()` en `iris`?

14. Utiliza las funciones específicas sobre la variable longitud del sépalo que calculan la media, la desviación típica, los cuantiles, la mediana, el mínimo y el máximo.

15. La función `cor()` permite calcular correlaciones. Se calcula el coeficiente de correlación lineal de Pearson por defecto. Es posible calcular otros coeficientes utilizando el argumento `method`. `?cor` para más información.

a) Calcula la correlación entre la longitud y anchura del sépalo.

b) Calcula la matriz de correlaciones considerando las cuatro primeras variables de `iris`, las que son cuantitativas.

16. La covarianza es un valor que indica el grado de variación conjunta de dos variables respecto a sus medias,

$$Cov(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{X})(y_i - \bar{Y})}{n}$$

La función `cov()` calcula la covarianza en R. Calcula la matriz de varianzas-covarianzas para las variables cuantitativas.

La función apply

La función `apply()` permite aplicar una función a través de una matriz, array o data frame, bien por filas o por columnas, dependiendo del argumento `MARGIN`.

```
apply(iris[,1:4], MARGIN=1, FUN=mean) # por filas
```

```
## [1] 2.550 2.375 2.350 2.350 2.550 2.850 2.425 2.525 2.225 2.400 2.700 2.500
## [13] 2.325 2.125 2.800 3.000 2.750 2.575 2.875 2.675 2.675 2.675 2.350 2.650
## [25] 2.575 2.450 2.600 2.600 2.550 2.425 2.425 2.675 2.725 2.825 2.425 2.400
## [37] 2.625 2.500 2.225 2.550 2.525 2.100 2.275 2.675 2.800 2.375 2.675 2.350
## [49] 2.675 2.475 4.075 3.900 4.100 3.275 3.850 3.575 3.975 2.900 3.850 3.300
## [61] 2.875 3.650 3.300 3.775 3.350 3.900 3.650 3.400 3.600 3.275 3.925 3.550
## [73] 3.800 3.700 3.725 3.850 3.950 4.100 3.725 3.200 3.200 3.150 3.400 3.850
## [85] 3.600 3.875 4.000 3.575 3.500 3.325 3.425 3.775 3.400 2.900 3.450 3.525
## [97] 3.525 3.675 2.925 3.475 4.525 3.875 4.525 4.150 4.375 4.825 3.400 4.575
## [109] 4.200 4.850 4.200 4.075 4.350 3.800 4.025 4.300 4.200 5.100 4.875 3.675
## [121] 4.525 3.825 4.800 3.925 4.450 4.550 3.900 3.950 4.225 4.400 4.550 5.025
## [133] 4.250 3.925 3.925 4.775 4.425 4.200 3.900 4.375 4.450 4.350 3.875 4.550
## [145] 4.550 4.300 3.925 4.175 4.325 3.950
```

```
apply(iris[,1:4], 2, mean) # por columnas
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
## 5.843333 3.057333 3.758000 1.199333
```

Valores missing

Muchas de las funciones que realizan operaciones algebraicas tienen el argumento `na.rm` que permite ignorar los valores *missing* cuando su valor es `TRUE`. De otra forma, devolverán `NA` siempre que alguno de los valores sea *missing*.

```
a <- c(1:4, NA, 6)
mean(a) # devuelve NA puesto que en a hay un NA
```

```
## [1] NA
```

```
mean(a, na.rm=TRUE)
```

```
## [1] 3.2
```

Contrastes de hipótesis

Los contrastes de hipótesis son útiles para evaluar la significación de nuestras hipótesis y ayudarnos en la toma de decisiones. La filosofía detrás de estos contrastes es determinar si nuestros datos aportan la suficiente evidencia para rechazar una hipótesis que asumimos como cierta. La elección de un tipo de contraste concreto va a depender de los datos que estemos manejando, de las asunciones que podemos hacer acerca de la distribución de nuestras variables y de las hipótesis que queramos contrastar. Ejemplos: contraste t-Student de una muestra, t-Student para dos muestras independientes, pareadas, etc. En R disponemos de muchas funciones para llevar a cabo contrastes de hipótesis.

Contraste t-Student para dos muestras independientes

Utilizamos el dataset `sleep` que contiene información sobre el efecto de dos fármacos para dormir, cada uno de ellos probado en 10 individuos. La variable de interés es el incremento de horas de sueño tras tomar el fármaco.

```
help(sleep)
```

```
data(sleep) # cargamos el data.frame
head(sleep)
```

```
##      extra group ID
## 1    0.7      1  1
## 2   -1.6      1  2
## 3   -0.2      1  3
## 4   -1.2      1  4
## 5   -0.1      1  5
## 6    3.4      1  6
```

Los dos grupos de tratamiento son independientes. Planteamos un contraste de igualdad de medias: contraste t-Student para dos muestras independientes:

```
test <- t.test(sleep[, "extra"] ~ sleep[, "group"])
test
```

```
##
## Welch Two Sample t-test --> las varianzas no son iguales por lo que no se usa t-student, sino welch
##
## data: sleep[, "extra"] by sleep[, "group"]
## t = -1.8608, df = 17.776, p-value = 0.07939
## alternative hypothesis: true difference in means between group 1 and group 2 is not equal to 0
## 95 percent confidence interval:
## -3.3654832  0.2054832
## sample estimates:
## mean in group 1 mean in group 2
##              0.75              2.33
```

Contraste chi-cuadrado

Este tipo de contraste está relacionado con el análisis de tablas de contingencia. Construimos una tabla de contingencia,

```
tbl <- matrix(c(14, 33, 7, 3), ncol = 2)
tbl
```

```
##      [,1] [,2]
## [1,]   14   7
## [2,]   33   3
```

Es una tabla de contingencia que cruza la información de dos variables: en columnas el tipo de coche (familiar o deportivo); y en filas el sexo del propietario.

```
colnames(tbl) <- c("Familiar", "Deportivo")
rownames(tbl) <- c("Hombre", "Mujer")
tbl
```

```
##      Familiar Deportivo
## Hombre      14         7
## Mujer       33         3
```

El contraste chi-cuadrado evalúa si las distribuciones de dos variables cualitativas son diferentes. Se basa en comparar las frecuencias observadas con las esperadas si no hubiera relación entre las variables. En este caso, ¿hay relación entre el sexo del conductor y el tipo de coche que conduce?

```
test <- chisq.test(as.table(tbl))
test

##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data:  as.table(tbl)
## X-squared = 4.1324, df = 1, p-value = 0.04207
```

Test de rangos con signo de Wilcoxon

Se trata de un contraste no paramétrico para evaluar la hipótesis de que no existen diferencias entre dos grupos de individuos dependientes.

Consideremos que tenemos una muestra de 12 estudiantes a los que les hacemos un examen de matemáticas al principio y al final del cuatrimestre.

```
dat2 <- data.frame(
  Beginning = c(16, 5, 15, 2, 14, 15, 4, 7, 15, 6, 7, 14),
  End = c(19, 18, 20, 17, 10, 17, 16, 19, 20, 9, 11, 18)
)
dat2
```

```
##      Beginning End
## 1           16 19
## 2            5 18
## 3           15 20
## 4            2 17
## 5           14 10
## 6           15 17
## 7            4 16
## 8            7 19
## 9           15 20
## 10           6  9
## 11           7 11
## 12          14 18
```

¿Hay diferencias en sus resultados? ¿Se obtienen mejores resultados al final del cuatrimestre?

```
test <- wilcox.test(dat2$End, dat2$Beginning, paired = TRUE, alternative = "greater")
test
```

```
##
## Wilcoxon signed rank test with continuity correction
##
## data:  dat2$End and dat2$Beginning
## V = 73, p-value = 0.004206
## alternative hypothesis: true location shift is greater than 0
```

Se puede acceder a los distintos elementos del objeto `test` que es una lista,

```
test$p.value
```

```
## [1] 0.004206118
```

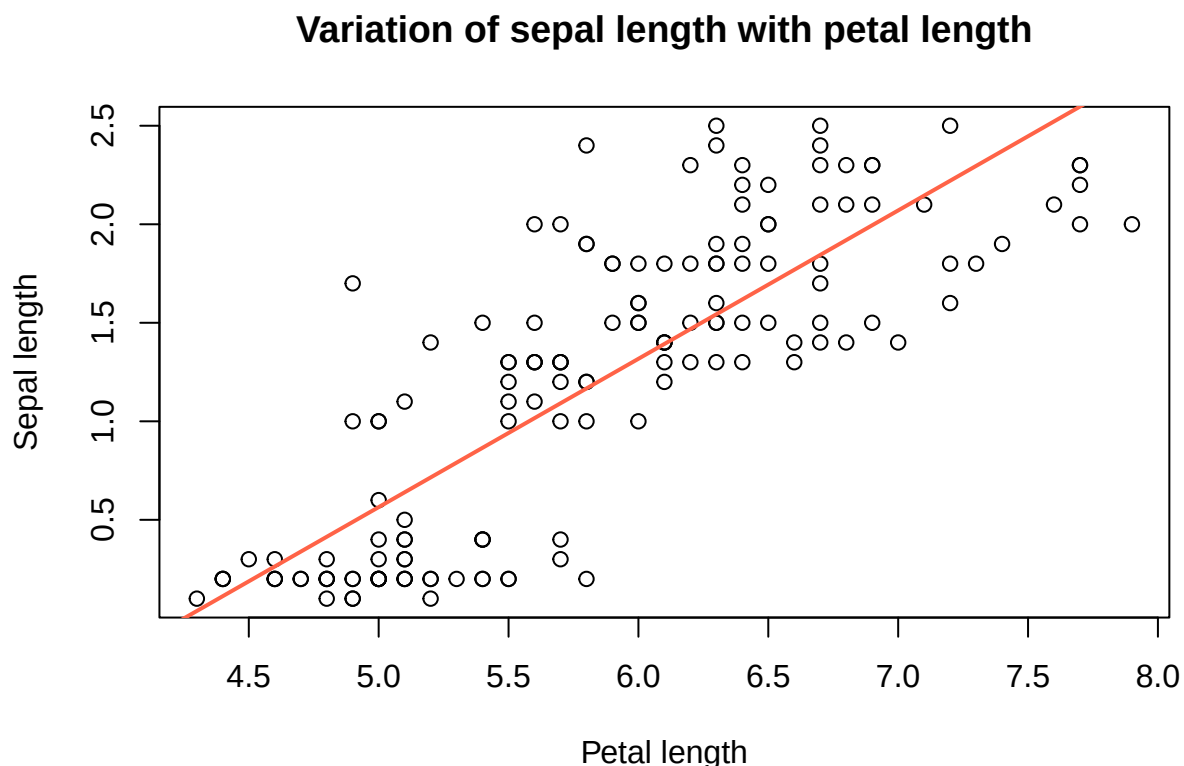
Visualización de los datos

En esta sección vamos a hacer un repaso de las formas más sencillas de visualizar los datos con funciones básicas de R. R es una herramienta muy potente para la construcción de gráficos e iremos viendo formas más sofisticadas a lo largo del curso.

Gráfico de dispersión

En el dataset iris, ¿hay relación entre las longitudes del pétalo y del sépalo?

```
plot(x=iris[, "Sepal.Length"], y=iris[, "Petal.Width"], xlab="Petal length", ylab="Sepal length",
     col="black", main="Variation of sepal length with petal length")
model<-lm(iris[, "Petal.Width"]~iris[, "Sepal.Length"]) # ajuste del modelo lineal
abline(model,col="tomato",lwd=2) # representación de la recta en el plot
```



Podemos asignar etiquetas a los ejes X e Y con los argumentos `xlab` e `ylab` respectivamente. También podemos asignarle un título con el argumento `main`. Podemos añadir líneas de tendencia a los gráficos de dispersión que resuman la relación entre las variables representadas con la instrucción `abline()`.

En lugar de puntos podemos representar líneas con el argumento `type="l"`. A veces representar puntos no es suficientemente informativo, especialmente cuando tratamos de representar procesos dinámicos. Por ejemplo, queremos representar la expresión de un gen a lo largo del tiempo o en diferentes concentraciones de un fármaco que estamos probando. Generamos 250 valores en tres situaciones diferentes asumiendo una distribución normal.

```
# 100 observaciones N(1,0.1), 100 N(2,0.1) y 50 N(3,0.1)
set.seed(1234)
genex <- c(rnorm(100, 1, 0.1), rnorm(100, 2, 0.1), rnorm(50, 3,0.1))
```

```
plot(genex, ylim=c(0,4), type='l', main="Line Diagram")
```

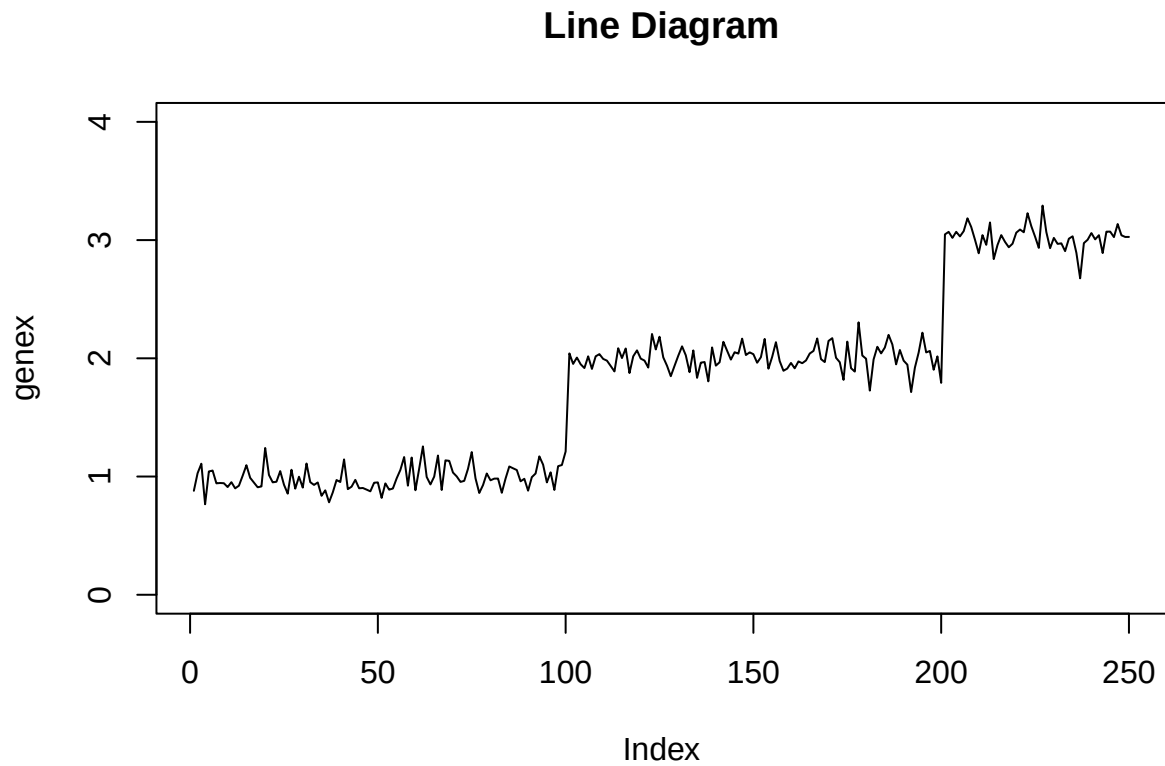
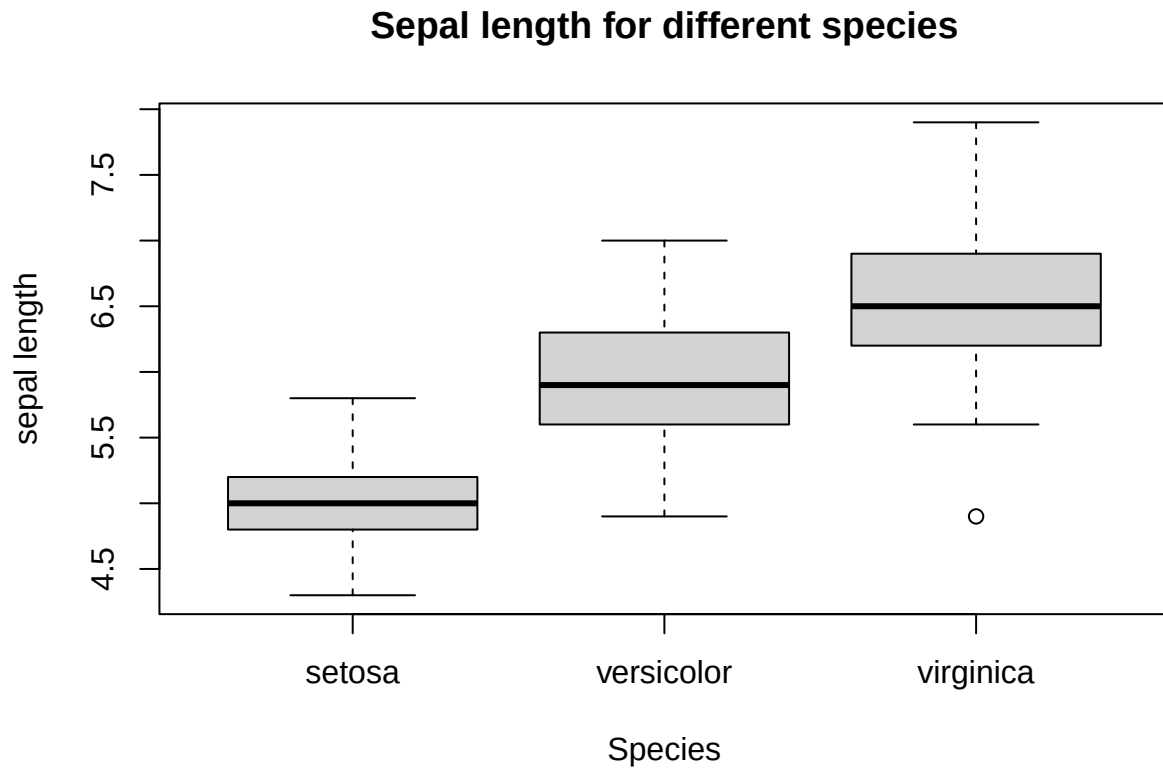


Diagrama de cajas

En el dataset iris, ¿hay diferencias entre especies en cuanto a la longitud del sépalo?

```
boxplot(Sepal.Length~Species, data=iris, ylab="sepal length",  
        xlab="Species", main="Sepal length for different species")
```

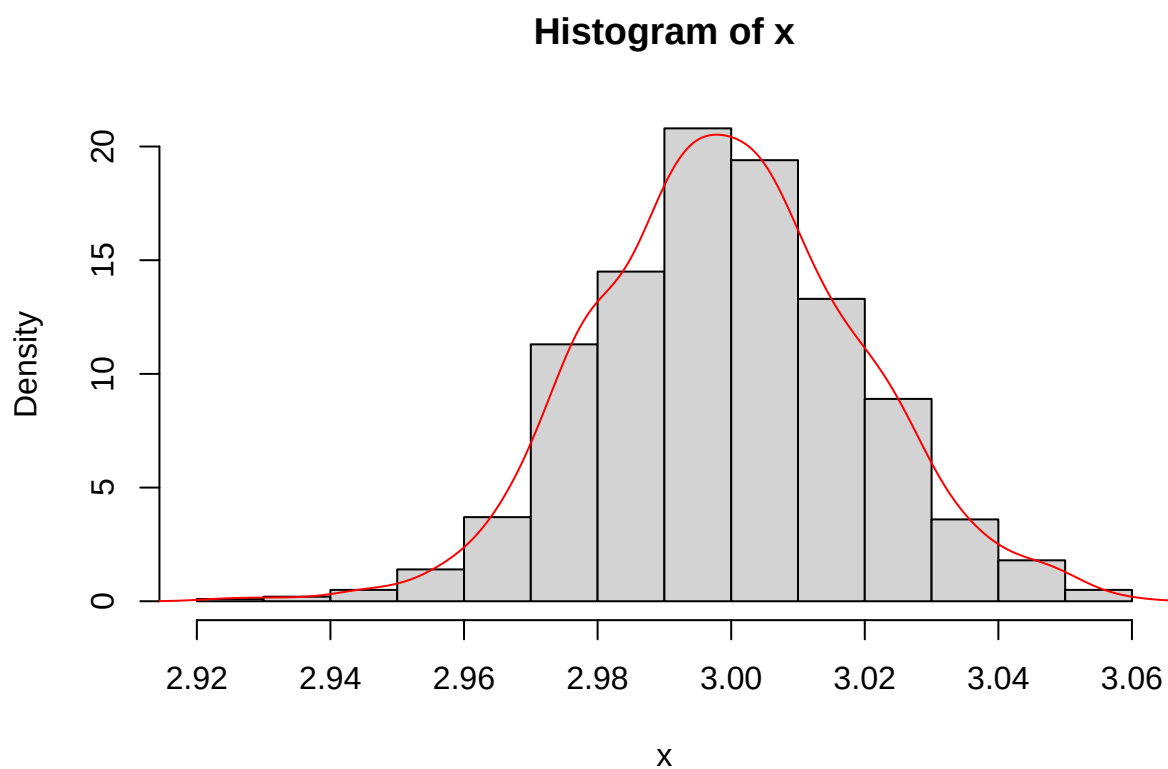


La función `boxplot()` lleva incorporado un algoritmo de detección de puntos atípicos u outliers, que representa en el gráfico como puntos más allá de los 'bigotes'. Se considera un outlier todo valor mayor que el tercer cuartil + 1.5 veces el rango intercuartílico o menor que el primer cuartil -1.5 veces el rango intercuartílico.

Histogramas

Sirven para visualizar la distribución de una variable cuantitativa.

```
set.seed(2345)
x <- rnorm(1000, 3, 0.02) # 1000 observaciones de una N(3,0.02)
hist(x, probability=TRUE)
lines(density(x), col="red")
```



Podemos añadir la función de densidad con la función `lines()`. En ese caso es importante utilizar el argumento `probability=TRUE` en la función `hist()`, para que la escala en la que se representa el histograma sea la de las frecuencias relativas y no las frecuencias absolutas.